CLIENT LAYER: React SaaS Web Dashboard

- **Tech:** React 18 • Vite • Tailwind CSS • Heroicons
- **Features:** Adaptive Socratic Chatbot with Step-by-Step Solver
- **Portals:** Student Mastery Dashboard & Teacher Class Analytics
- **Auth:** JWT Bearer Session Storage, Role Guard Routing

HTTP / REST (JSON)

CLIENT LAYER: Automated Judges CLI Contract

- **Command:** python main.py --json "<query>"
- **Compliance:** Emits EXACTLY 1 raw JSON line containing {answer, trace}
- **Modes:** --dry-run for offline mock, --debug for step inspection
- **Output:** All diagnostic logs suppressed or redirected to stderr

Direct CLI Invocation

API & ORCHESTRATION GATEWAY: FastAPI Layer (server/app.py)

- **Endpoints:** /api/tutor/chat • /api/auth • /api/students • /api/curriculum • /api/health
- **Security:** Native Bcrypt Password Hashing • Student Data Isolation Guarantees • CORS Middleware
- **Static Host:** Mounts production SPA build (frontend/dist) directly at root / with client-side fallback
- **Traceability:** Logs every user thought/action session to database AgentTrace table for post-audit

Session State & Prompt

REASONING CORE: ReAct Agent (agent/core.py)

- **Architecture:** Thought -> Action -> Action_Input -> Observation Loop
- **LLM Engine:** Google Gemini via official google-genai SDK
- **Model:** gemini-3.5-flash-lite (Active & Production-ready)
- **Resilience:** Auto-repetition breaking, JSON parsing self-correction

Dispatches Tools

INTELLIGENT TOOL SUITE: 8 Pedagogical Tools

- **Diagnostics:** student_performance • misconception_tool
- **Curriculum:** curriculum_retrieval (grounded chunk search)
- **Pedagogy:** adaptive_tutor_tool • knowledge_graph_tool
- **Remediation:** recovery_planner_tool • assessment_tool • intervention_tool

SQLAlchemy ORM

PERSISTENCE TIER: Dual-Engine SQLAlchemy Database (database/)

- **Dual Support:** Zero-config SQLite for standalone evaluation & PostgreSQL for enterprise cloud scale
- **Core Models:** User • TeacherClass • StudentEnrollment • Subject • Topic • TopicPrerequisite
- **Pedagogy Tables:** TopicMastery • Misconception • StudentAssessment • StudyPlan • InterventionHistory
- **Knowledge Engine:** CurriculumDocument • CurriculumChunk • AgentTrace • StudentLearningMemory

WORKFLOW 1: SOCRATIC AI TUTORING & PROBLEM SOLVING

End-to-End Inquiry, Retrieval & Guided Pedagogical Reasoning

STAGE 1: Student Inquiry

- **Trigger:** Student asks question in Chat Tutor
- **Modes:** Concept / Problem / Exam Review
- **Payload:** {message, subject_id, mode}
- **Identity:** JWT token verified by FastAPI
- **Context:** Loads last 6 conversation turns

Auth & Context →

STAGE 2: Mastery & Misconception Lookup

- **Tool:** student_performance & misconception_tool
- **Analysis:** Fetches student current topic mastery
- **Risk Check:** Identifies active misconception flags
- **Calibration:** Sets cognitive difficulty level
- **Isolation:** Guarantees student cannot view peer data

Mastery Score →

STAGE 3: Grounded Curriculum RAG

- **Tool:** curriculum_retrieval
- **Search:** Queries vetted textbook & lecture chunks
- **Source:** Biology, Chemistry, Math, Physics docs
- **Anti-Hallucination:** Injects exact course excerpts
- **Fallback:** Structured knowledge base grounding

↓

Grounded Curriculum Chunks + Student State →

CORE PEDAGOGY RULE: Knowlix Never Gives Unearned Answers

The agent assesses prior mastery, fetches textbook chunks, and guides students step-by-step.

↓

STAGE 4: ReAct Pedagogical Loop

- **Orchestrator:** agent/core.py with Google Gemini
- **Thought:** Decides pedagogical scaffolding needed
- **Rule:** DO NOT spoon-feed direct answers
- **Socratic Prompt:** Asks leading guiding question
- **Step Scaffolding:** Breaks complex problems into mini-steps

Thought Sequence →

STAGE 5: Trace & Memory Logging

- **Audit Log:** Saves full trace in AgentTrace
- **Memory Store:** Saves student response pattern
- **Telemetry:** Tracks latency, tool calls, token count
- **ersistence:** Committed to SQLite/Postgres DB
- **Observability:** Inspectable by teachers & judges

Formatted Payload →

STAGE 6: Interactive Response Delivery

- **Client UI:** React ChatTutor component
- **Rendering:** Rich Markdown, math & code formatting
- **Chips:** Generates 3 interactive follow-up pills
- **eedback:** Thumbs up/down feedback widget
- **Loop Closure:** Awaits next student problem attempt

WORKFLOW 2: PREREQUISITE & MISCONCEPTION RECOVERY

Automated Knowledge Graph Traversal & Diagnostic Remediation

DAG PREREQUISITE GRAPH (knowledge_graph_tool)

Graph Nature: Directed Acyclic Graph of Subject Topics
Dependency 1: Basic Chemistry ---> Macromolecules ---> Enzymes
Dependency 2: Cell Structure ---> Membrane Transport ---> Osmosis
Dependency 3: Light Reactions ---> Calvin Cycle ---> Carbon Fixation
Bottleneck Logic: A student failing an advanced topic is rarely weak
Root Cause: in the advanced topic itself; they lack prerequisite mastery.
Tool Action: Traverses parents recursively to locate lowest unmastered node.

MISCONCEPTION CATALOG (misconception_tool)

Target Detection: Diagnoses high-frequency cognitive errors
Example 1: 'Plants do photosynthesis, animals do respiration' [FALSE]
Correction 1: Plants possess mitochondria and respire 24/7.
Example 2: 'Mass of trees comes from soil minerals' [FALSE]
Correction 2: 95%+ of dry plant biomass is fixed from atmospheric CO2.
Remediation: Contrasts intuitive misconception with empirical reality
Verification: Issues micro-quiz targeting the exact boundary condition.

Unmet Prerequisite Node ↓ **Diagnosed Error Model** ↓

AUTOMATED ADAPTIVE RECOVERY PIPELINE (recovery_planner_tool & intervention_tool)

Step 1 - Detection: Student fails assessment questions or exhibits confusion during chat on target topic.
Step 2 - Graph Walk: knowledge_graph_tool walks upstream: checks prerequisite nodes for mastery threshold (< 0.70).
Step 3 - Diagnosis: misconception_tool checks student answers against known false cognitive models.
Step 4 - Prescription: recovery_planner_tool generates a personalized 3-phase StudyPlan with sequenced StudyPlanTasks:
- **Phase A:** Remediate missing prerequisite foundation chunk (e.g. review ATP currency before Calvin cycle)
- **Phase B:** Directly refute identified misconception using visual comparison counter-example
- **Phase C:** Re-assess target topic with calibrated formative quiz (assessment_tool)

Step 5 - Intervention: intervention_tool records action in InterventionHistory and alerts Teacher Dashboard if plateau persists.

COMMAND LINE INTERACTION SPECIFICATION

Judge Command: python main.py --json "<query>"
Strict Rule 1: STDOUT contains EXACTLY 1 raw JSON line. No logs.
Strict Rule 2: No banners, no ANSI colors, no markdown formatting on stdout.
Strict Rule 3: Exit code 0 on successful processing.
Flag --dry-run: Runs entirely offline without requiring live API keys.
Flag --max-steps: Caps max tool iterations (default: 8) to prevent loops.
Flag --debug: Directs step-by-step ReAct traces exclusively to stderr.

Parses & Formats →

OUTPUT JSON SCHEMA & VALIDATION

Root Object: { 'answer': string, 'trace': list }
answer Field: Comprehensive, accurate pedagogical answer to query
trace Field: List of execution step objects [step_1, step_2, ...]
Step Object: { 'step': int, 'thought': str, 'action': str, 'action_input': dict/str, 'observation': str }
Validation: Parsable by json.loads(stdout.strip()) without error
Zero Hallucination: Grounded in seeded curriculum or tool outputs.

Verified by ↓

Observed on CLI ↓

JUDGE VERIFICATION TEST RESULTS

Suite: python -m unittest discover tests
Execution: Ran 20 unit tests in 0.971s — 100% PASSING (OK)
Coverage Areas:
- **Test 1-5:** ReAct loop termination, self-correction, dry-run
- **Test 6-10:** 8 Specialized pedagogical tool execution & schemas
- **Test 11-14:** Database seeding, SQLite/PostgreSQL connection
- **Test 15-17:** FastAPI endpoints & native Bcrypt authentication
- **Test 18-20:** Student isolation security & curriculum chunking

SAMPLE TEST RUNS VERIFIED ON REPO

Query 1: python main.py --json "Explain photosynthesis"
Result 1: answer: 'Photosynthesis is the process... [C4 / Light]'
Trace 1: Called curriculum_retrieval -> returned vetted text
Query 2: python main.py --json "Which topics is S1 weak in?"
Result 2: answer: 'Student S1 is weak in Cell Respiration'
Trace 2: Called student_performance -> fetched TopicMastery < 0.7
Query 3: python main.py --json --dry-run "test"
Result 3: Mock fallback executed cleanly with valid JSON schema

DATA ARCHITECTURE & ENTITY RELATIONSHIP MODEL — 17 Relational Entities in Dual-Engine SQLAlchemy Store

IDENTITY & CLASSES

User
- **id:** Integer PK
- **email:** String(255) Unique
- **password_hash:** String
- **full_name:** String(100)
- **role:** student / teacher
- **created_at:** DateTime

TeacherClass
- **id:** Integer PK
- **teacher_id:** FK -> User.id
- **name:** String(100)
- **grade_level:** String(20)

StudentEnrollment
- **id:** Integer PK
- **student_id:** FK -> User.id
- **class_id:** FK -> Class.id

KNOWLEDGE GRAPH

Subject
- **id:** Integer PK
- **name:** String(100)
- **code:** BIO / CHEM / MATH

Topic
- **id:** Integer PK
- **subject_id:** FK -> Subject.id
- **name:** String(150)
- **difficulty_level:** 1 - 5

TopicPrerequisite
- **id:** Integer PK
- **topic_id:** FK -> Topic.id
- **prerequisite_id:** FK -> Topic

TopicMastery
- **id:** Integer PK
- **student_id:** FK -> User.id
- **topic_id:** FK -> Topic.id
- **mastery_score:** Float 0.0-1.0

PEDAGOGY & RECOVERY

Misconception
- **id:** Integer PK
- **topic_id:** FK -> Topic.id
- **misconception_text:** Text
- **correction_strategy:** Text

StudentAssessment
- **id:** Integer PK
- **student_id:** FK -> User.id
- **score:** Float
- **date_taken:** DateTime

StudyPlan & Task
- **id:** Integer PK
- **student_id:** FK -> User.id
- **status:** active/completed
- **task_type:** review/quiz

InterventionHistory
- **action_taken:** Text
- **trigger_reason:** Text

KNOWLEDGE & TRACE

CurriculumDocument
- **id:** Integer PK
- **title:** String(200)
- **subject_id:** FK -> Subject.id
- **source_file:** String

CurriculumChunk
- **id:** Integer PK
- **document_id:** FK -> Doc.id
- **chunk_index:** Integer
- **content:** Text
- **token_count:** Integer

AgentTrace
- **session_id:** String(64)
- **query:** Text
- **thoughts:** JSON / Text
- **execution_time_ms:** Int

StudentLearningMemory
- **key_fact:** Text